Detecting Self-Admitted Security Debt in Software Projects using Natural Language Processing

Blake Harris
April 25, 2025

***Abstract*—Self-Admitted Security Debt (SASD) is difficult to identify in software projects as code comments, commit messages, and issue trackers often contain unstructured text that is hard to monitor. Existing tools to detect Technical Debt usually focus on general code quality and are not specifically targeted at security-related debt. Unidentified security debt poses a threat to software integrity, potentially exposing software to vulnerabilities and exploits. The research addresses the feasibility of a tool which uses Natural Language Processing (NLP), specifically GPT-3.5-Turbo, to automatically detect SASD from GitHub repository artifacts and map detected instances to Common Weakness Enumerations (CWEs) to categorize vulnerabilities. Experiments conducted demonstrated that the developed NLP-based solution significantly outperformed traditional keyword-based methods, achieving near-perfect recall and substantially improved F1-scores in detecting SASD. Additionally, while the NLP method achieved a moderate CWE mapping accuracy average around 35%, it represented a considerable improvement over the baseline method, providing valuable insights for vulnerability prioritisation. Despite dataset limitations and NLP model variability, this research confirms the practical value of integrating NLP into software development workflows, enabling proactive and efficient remediation and management of security risks, thus promoting more secure and resilient software systems.**

***Index Terms*—Self-Admitted Security Debt (SASD), Self-Admitted Technical Debt, Natural Language Processing (NLP), Common Weakness Enumerations (CWEs), Software Security.**

I. Introduction

A. Background

Security debt is a specific form of Technical Debt, focusing on incomplete security-related tasks that are deferred to expedite development, potentially compromising long-term system security. Security debt is essentially security-related work that is not completed at an optimal time to mitigate known or unknown security issues in a system [1]. When developers explicitly acknowledge suboptimal code design in projects, we refer to this as "Self-Admitted Technical Debt". The acknowledgments commonly appear in source code comments, Git commit messages, and issue tracker entries [2]. Technical debt parallels financial debt, with savings gained through shortcuts. Much like monetary contexts, TD must be repaid, and accrues interest in different forms - often time, money, and resources [3].

Self-Admitted Security Debt is a subset of technical debt where developers acknowledge security-related weaknesses that could introduce software vulnerabilities. Understanding and addressing SASD is vital for maintaining robust security practices in software development [2], [4]. However, Security Debt being a new concept, means there is little or no support for automation for developers who want to efficiently reduce the amount of debt [1].

B. Harris is with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, UK, e-mail: bharris06@qub.ac.uk

B. Motivation and Context

Detecting security debt is particularly difficult due to the unstructured nature of text in code comments and commit messages. While tools exist for detecting general technical debt, such as SonarQube and Jira, these focus primarily on the quality of code without targeting security-related issues [2]. This is an obvious problem, as unidentified security debt compromises software integrity and creates entry points for exploits, thus leaving software open to attack [2], [4].

Existing technical debt detection tools do not specifically address security-related debt. The gap in current research overlooks software security, as no automated solutions exist that identify SASD and map findings to established vulnerability frameworks such as Common Weakness Enumerations (CWEs) [2], [4]. Additionally, in prior work it was highlighted that "multi-source studies at the interface between SATD and security are missing in the current literature", and "secret scanning tools should extend their scope with SSATD identification features to assist developers" [2]. This shows the limitations of current tools in detecting and categorizing security-related technical debt. Security-related comments in code or Git artifacts that are unaddressed can inadvertently guide attackers to vulnerabilities, effectively providing a roadmap to potential exploits.

C. Problem Statement

Despite increasing awareness of technical debt, there is no widely adopted automated solution capable of systematically detecting and categorizing security-specific self-admitted technical debt (SASD). Particularly, the unstructured nature of repository artifacts presents a significant challenge in identifying security weaknesses. As a result, critical security concerns may go unnoticed, thus leaving software vulnerable to attack or exploitation. Furthermore, by failing to map potential security issues to known vulnerability frameworks such as CWEs, organizations miss the opportunity to prioritize and remediate issues efficiently and effectively.

D. Research Aim and Approach

This research aims primarily to determine the feasibility of developing an NLP-based tool to detect and identify SASD within software projects. The tool will use the GitHub API to collect data from source code comments, Git commit messages, and issue trackers. This data will be pipe-lined into an NLP model API to detect security-related phrases or patterns. The findings will be mapped to CWEs and through this, the tool will categorize vulnerabilities and prioritize them during code reviews and maintenance.

The research is expected to culminate in a software solution that accurately identifies SASD and provides developers with actionable insights to manage security-related debt effectively. The tool will enable development teams to prioritize and address security debt during codebase reviews, thus leading to early detection and rectification of vulnerabilities. This solution addresses the current gap in SASD detection, contributing to a more secure and resilient software development environment.

E. Research Questions

In response to the problems identified above, we pose the following questions:

1) *How can Natural Language Processing (NLP) be applied to automatically detect security-related self-admitted technical debt (SASD) from repository artifacts?*
2) *How can NLP Classification map detected SASD instances to CWEs to categorize security vulnerabilities?*

II. Literature Review

A. Key Concepts and Definitions

Technical Debt (TD) has long been described as suboptimal development choices that defer concrete design decisions to expedite delivery, accruing "interest" in the form of additional maintenance overheads or refactoring needs [3]. Within this lies the subcategory of Security Debt, which focuses on deferred or incomplete security-related tasks that may expose software systems to vulnerabilities [1]. When developers explicitly acknowledge their shortcomings - such as in code comments, Git commit messages, or issue trackers - we refer to this as Self-Admitted Technical Debt [2], [4]. In the context of security, we therefore refer to this as Self-Admitted Security Debt (SASD) [2], [4]. As these issues can introduce security-critical risks if overlooked, SASD represents a pressing subcategory of technical debt [1], [2].

Huopio [1] highlights that "a large amount of SD is a potentially massive liability to customers", reinforcing the real-world repercussions of delaying security fixes. This observation broadens the scope of SASD, suggesting that unaddressed issues can accumulate into a significant threat for both software maintainers and end-users.

B. Theoretical and Conceptual Frameworks

Existing frameworks tend to address technical debt more broadly, with few aimed at security-specific issues. Rindell et al. [5] proposed the "ACT-S" framework, which classifies security debt into four primary subtypes - requirement, architectural, code, and testing - and integrates security engineering into technical debt management (TDM) activities. This approach provides a structure for requirement reviews, architecture assessments, and automated security testing to detect and mitigate debt early. Moreover, the framework highlights how some organisations may be financially driven to carry security debt if "repayment" can be offloaded to customers or delayed. They note, "some organisations may be inclined to take the debt, as the repayment is done by the customers in the form of lowered usability" [5]. Despite this insight, ACT-S doesn't integrate automated Natural Language Processing (NLP) methods specific to SASD detection.

Other conceptual approaches, such as those embedded in general-purpose tools (e.g., SonarQube, Jira), focus on improving code quality but don't systematically target the text-based security hints that developers leave behind in code comments or Git artifacts. This gap leaves software teams relying on manual or generalised scanning, thus undermining the capacity to catch subtle yet potentially critical security signals [2], [4]. While frameworks such as ACT-S underline the importance of integrating security reviews into each development phase, there still remains a need for a method targeted specifically at capturing and interpreting developers' self-admitted references to security debt.

Similarly, analogous to a technical debt ledger, Greer et al. [6] propose a "security debt ledger" concept to track "unpaid debts" and highlight the dangers of postponing security tasks - thus reflecting how quickly incomplete security work can accumulate and pose systemic risks.

C. Approaches to Detecting Technical and Security Debt

A variety of approaches have been investigated for detecting both Self-Admitted Technical debt and Security debt:

1) **Manual and Keyword-Based Approaches**
   Many studies emphasize the benefits of manual review for capturing contextual nuances [5], [7]. Potdar and Shihab [7] explored manual classification and keyword searches, revealing that time pressures and code complexity aren't strong predictors of SATD, and that "2.4 - 31.0% of the files in a project contain self-admitted technical debt". They noted that only 26.3 - 63.5% of this debt is removed once introduced, thus implying a long-term accumulation of debt. They further observed that only 26.3 - 63.5% of that debt is addressed and removed, implying a long-term accumulation and an even greater risk to security.
   Fahmawi et al [8] studied manual code reviews to detect security-related vulnerabilities. They underscored that developers often respond positively once vulnerabilities are highlighted, but that human error and the sheer scale of codebases impose practical limitations. These manual or keyword-only strategies risk overlooking subtle phrasing or context-dependent disclaimers, especially in large, fast-evolving projects [2].
2) **NLP-Driven SATD and Early SATD Detection**
   Maldonado et al. [4] were among the first to apply advanced NLP techniques to automatically detect SATD,

using maximum entropy classifiers to flag design and requirement debt. Their experiments suggested that "with as little as 23% of the available data, one can achieve 90% of the maximum F1-measure in detecting [debt]", highlighting the potential efficiency of data-driven approaches. Despite their model demonstrating improved detection accuracy over simply keyword matching, it didn't explicitly address security-specific debt. This gap is exactly where SASD detection remains largely unexplored.

Ferreyra et al. [2] built on these approaches by mapping SATD to security vulnerabilities via the Common Weakness Enumerations (CWEs) framework. Their work revealed "25 different types of CWEs" arising from SATD, eight of which appear among MITRE's Top 25 most dangerous. They observed that "code/design and requirement debt is concentrated around issue sections, whereas defects are often self-admitted through code comments", and that "commit messages tend to gather a majority of the test debt cases" [2]. While their analysis underscores the importance of detecting security vulnerability indicators from diverse software artifacts, it doesn't offer a fully automated, end-to-end system for SASD detection.

3) **Security Debt in Practice**

Following the detection stage, Huopio [1] and others have highlighted the implications of unaddressed security debt, noting how only high-profile vulnerabilities might receive fixes while "low priority issues are left unhandled". These lower-level issues can accumulate into a compound risk that exceeds the threat of any one single, high-severity bug. Rindell et al. [5] similarly observed that "less than one out of ten bugs or flaws are caught in testing", and "over a quarter of these gets through all quality controls, to be found during the operations phase". These findings highlight how frequently security debt is overlooked during development and testing.

Additionally, in business-oriented or fast-growing projects, there can be little enthusiasm to pause development for security refactoring [6]. Greer et al. [6] propose ideas such as a "security debt ledger" and leveraging conversational chatbots to track security concerns earlier in the development process. Their case examples emphasize how delayed prioritization of security can lead to severe breaches and public trust erosion if vulnerabilities are only recognized post-deployment.

D. Gaps in Current Research

Although researchers have explored automated detection of general SATD with notable success, there are far fewer studies that focus on security-specific debt. Tools such as SonarQube or general "secret scanning" solutions often lack SASD detection capabilities beyond trivial keyword checks [2]. As Ferreyra et al. emphasize, "multi-source studies at the interface between SATD and security are missing in the current literature", and "secret scanning tools should extend their scope with SSATD identification features to assist developers" [2].

Unaddressed security comments could inadvertently guide attackers toward vulnerabilities, as exposing these details in code or Git artifacts can act as a roadmap to potential exploits — a risk also implied by Fahmawi et al [8] in their discussion of vulnerabilities that sneak through manual reviews.

Another gap lies in the mapping of identified debt to actionable frameworks, such as CWEs [2]. While there is evidence to suggest that attaching recognized vulnerabilities to CWE categories can help developers to prioritize remediation, there is no widely adopted, fully automated pipeline that exists which detects SASD and contextualizes it using established security references.

Recently, Mock et al. proposed a unified approach that uses both NLP and NL-PL (Natural Language Programming Language) techniques to detect vulnerabilities and associated self-admitted debt in source code comments. They state that their approach can "Accurately identify and better understand the semantics of self-admitted technical debt (SATD) by leveraging NLP and NL-PL approaches to detect vulnerabilities and the related SATD", and also acknowledge that "to the best of our knowledge, no tool leverages both concepts by connecting them besides our own tool" [9]. However, their work is still at the prototype stage without a comprehensive empirical evaluation across different repository artifacts or an end-to-end mapping into established vulnerability frameworks such as the Common Weakness Enumerations — precisely the gap this research aims to fulfil.

Finally, literature indicates an organizational and cultural dimension: many teams — even those with experienced developers — take on technical or security debt to meet pressing deadlines [5], [8]. Given this, an automated solution that highlights SASD for timely remediation could be pivotal in preventing compounding security risks over multiple versions or releases.

E. Synthesis of Themes

Across studies, a few primary themes emerge:

- **Context Matters:** Security debt indicators often appear in unstructured text - commit messages, issue trackers, and code comments - and require nuanced handling beyond simple keyword detection.
- **NLP Efficacy:** Advanced machine learning and NLP models can outperform manual or rule-based methods for detecting subtle references to technical debt [4]. This advantage is perhaps more pronounced for security-related indicators, which may be more linguistically diverse.
- **Underexplored Security Angle:** While SATD research has expanded, SASD remains relatively understudied, leaving a gap that an automated, text-centric solution could fill [2], [4].
- **Potential Impact on Vulnerability Management:** Integrating CWE mappings can transform raw SASD detections into actionable security insights for developers, guiding priorities in code reviews and TDM processes.

F. Relevance to Research Questions

Developing an NLP-based tool that systematically identifies security-specific self-admitted dent across multiple software artifacts (code comments, commit messages, issue trackers), this research aims to address the documented gap in security-focused technical debt detection [2], [4], [8].

Specifically, it seeks to answer two core questions:

1) *How can Natural Language Processing (NLP) be applied to automatically detect security-related self-admitted technical debt (SASD) from repository artifacts?*
2) *How can NLP Classification map detected SASD instances to CWEs to categorize security vulnerabilities?*

Furthermore, the proposed integration of CWE mapping would offer tangible remediation pathways and help developers prioritize vulnerabilities based on severity - a need highlighted often [2], [5]. In doing so, the research answers calls to prevent unacknowledged or low-priority security issues from becoming major breaches [1].

G. Summary

The studies reviewed consistently point to a critical need for specialized, automated approaches that detect and categorize security-focused debt. These studies also validate the potenial of NLP in enhancing detection accuracy for SATD, but they also recognize that SASD remains largely unaddressed by existing tools and frameworks. This research aims to address the gap by leveraging NLP to capture developer-acknowledged security-related shortcomings and map them to known vulnerability frameworks.

III. Methodologies

A. Research Design

This research uses a mixed-methods approach, combining quantitative analysis of detection rates with qualitative evaluations of Self-Admitted Security Debt (SASD). By utilizing GPT-3.5-Turbo for Natural Language Processing (NLP), this research investigates whether automated analysis of code comments, commit messages, and issue trackers can accurately uncover developer-acknowledged security vulnerabilities.

*1) **Research Questions and Hypotheses**:* The research is driven by two main research questions (RQs):

(a) **RQ1:** *How can Natural Language Processing (NLP) be applied to automatically detect security related self-admitted technical debt (SASD) from repository artifacts?*
- **H0:** The NLP-based software does NOT significantly outperform a keyword-based approach in identifying SASD-related comments.
- **H1:** The NLP-based software significantly improves the identification of SASD-related comments compared to a keyword-based approach.

(b) **RQ2:** *How effectively can NLP Classification map detected SASD instances to CWEs to categorize security vulnerabilities?*
- **H0:** The NLP-based classification does not achieve a significantly different mapping accuracy from that of random assignment in categorising SASD instances.
- **H1:** The NLP-based classification significantly improves the mapping accuracy of SASD instances to CWEs, relative to random assignment.

B. Data Collection Strategy

*1) **Dataset Description**:* The dataset used for the experiments is a manually labelled dataset of Self-Admitted Technical Debt from open-source Java projects, created by Everton da Silva Maldonado. It comprises approximately 62,000 comment blocks, each enriched with metadata - such as project information (e.g. apache-ant-1.7.0), classification information (e.g. DEFECT), and the associated source code comment. Although originally designed for general SATD detection, the real and diverse data provides a solid basis for investigating security debt in real-world scenarios. However, while the dataset reliably represents SATD overall, it contains a limited amount of security-specific debt - a challenge that was encountered during the experiments.

*2) **Data Preprocessing**:* The dataset was converted from CSV format into a Python code file to enable seamless integration with the backend functionality of the developed software tool. A stratified sampling technique was applied to extract 1% of the full dataset, ensuring a representative sample of the original data distribution. To address the datasets intrinsic weakness in terms of security-related debt, Synthetic Data Augmentation (SDA) techniques were employed. Specifically, 20 synthetic self-admitted security debt comments were generated and incorporated into the stratified sample. Research by Wei and Zou [10] indicates that SDA techniques in text classification tasks can yield performance improvements of approximately 0.8% on full datasets and up to 3.0% on smaller training sets by increasing data diversity without compromising semantic integrity. These quantitative improvements support the argument that incorporating synthetic examples does not negatively affect experimental results, but instead enhances model generalisation in text classification, thus ensuring experimental outcomes are robust and reliable.

C. NLP-Based Detection Approach

*1) **Model Selection Rationale**:* For rapid integration and cost-effectiveness, GPT-3.5-Turbo is used via OpenAI's API. Beyond the economic advantages, the model provides strong contextual understanding, which is essential to identify nuanced references to deferred security work. Its ability to process unstructured text makes it well suited for analyzing Git artifacts, where developers may implicitly acknowledge security debt.

*2) **Prompt Engineering**:* Custom prompt engineering allowed for tailored GPT-3.5-Turbo responses when detecting SASD. The prompts instruct the model to:

- Analyze input messages or method bodies for indicators of security-related technical debt;
- Respond with a clear "Yes" or "No" alongside a brief explanation;
- Where security debt is detected, further map the analysis to CWE categories via an additional prompt.

Iterative refinements of these prompts have been vital to ensure the model's responses align with the security context and reducing false positives.

D. Experiment Setup and Evaluation Strategy

To evaluate the proposed approach, two sets of experiments are conducted - one focusing on SASD detection (RQ1) and another on CWE mapping (RQ2).

*1) **RQ1: SASD Detection Experiment**:*

(a) **Data Input and Processing:** The augmented, stratified sample of the full dataset is used as the input for the experiment. Detection outputs from both the baseline and NLP-based approaches are stored in a combined CSV file for unified analysis.

(b) **Detection Approaches:**

- **GPT-based Detection:** Each comment is analyzed via GPT-3.5-Turbo to flag potential SASD instances.
- **Baseline Keyword-Based Detection:** A predefined set of security terms is applied to the same comment as a baseline comparative.

(c) **Evaluation Metrics for RQ1:**

- **Recall:**

$$\frac{TruePositives}{(TruePositives + FalseNegatives)}$$

- **F1-Score:**

$$\frac{TruePositives}{TruePositives + \frac{1}{2}(FalsePositives + FalseNegatives)}$$

- Additional performance metrics collected include precision, false detection rate, specificity, and accuracy.
- Recall and F1-score serve as the primary metrics for comparing the effectiveness of two approaches.

(d) **Procedure:**

- The experiment is conducted over 10 runs to ensure robust results.
- For the majority of runs, fixed random seeds (42, 99, and 123) are used for sampling; on runs 3, 6, and 9, randoms seeds are used to capture variability.
- During each run, three samples are extracted from the combined results file for manual verification.
- Manual verification is performed on these samples, and aggregated results are analyzed to support or refute the null hypothesis (H0) for RQ1.

*2) **RQ2: CWE Mapping Experiment**:*

(a) **CWE Assignment:**

- During the detection experiment, the API response will include the model's proposed CWE mapping for each comment it identifies as security debt; these mappings are stored for use in this experiment.
- For the baseline approach, the combined detection results CSV is examined for true detections, and a keyword-based method is again used to map these comments to predetermined CWE categories.

(b) **Reference Validation:**

- Manual verification is performed on samples, extracting only comments that have been confirmed as security debt (ground truths) from both approaches.
- For each true SASD comment, two or three of the most suitable CWE categories are manually selected. A keyword-based search is applied to both approaches mappings to identify a matching CWE.

(c) **Evaluation Metrics for RQ2:**

- **CWE Mapping Accuracy:**

$$\frac{Correct\ CWE\ Assignments}{Total\ SASD\ Instances\ Mapped} \times 100$$

(d) **Procedure:** After NLP-based detection, the subset of SASD instances is passed to the CWE mapping module. The resulting classifications are then analyzed to determine if the mapping accuracy meets the 75% threshold required to support H1 for RQ2.

E. Limitations

- **Model Variability:**
GPT-3.5-Turbo, like other large language models (LLMs), is sensitive to prompt phrasing and contextual inputs. Minor variations in the prompt can lead to different outputs, and in some cases the model may even "renege" on previous judgement when given similar inputs in different contexts. This variability can lead to inconsistencies across different experimental runs, thus affecting repeatability and reliability. To mitigate these issues, our experimental design incorporates fixed random seeds in most runs and multiple iterations to capture and average out variability.

- **Dataset Limitations:**
The primary dataset originates from a manually labeled SATD dataset originally designed for general technical debt detection. Although it has been augmented with synthetic security-specific debt examples, the proportion of security-related entries is relatively low. This may limit the model's ability to generalise to broader contexts and could artificially boost performance metrics when evaluated on the augmented data. Furthermore, reliance on projects such as Ant, ArgoUML, and others — which

may no longer be fully active — raises questions about the representativeness of the dataset with regards to modern software development practices.

- **Manual Verification:**
  Due to time and resource constraints, only a subset of detected instances undergoes manual human verification. This limited scope may introduce sampling bias, potentially skewing the evaluation of detection and mapping metrics. The degree to which these samples represent the entire dataset is a point of concern that could affect the conclusions of the research.

- **Impact on Outcome Reliability:**
  The combination of both model variability and dataset limitations means that while experimental outcomes can offer valuable insights into the feasibility of our proposed solution, these outcomes should be interpreted with caution. Variability in results across different runs and possible limitations in data representativeness require careful discussion in terms of the generalisability of our findings. Future work should consider the inclusion of more current and diverse datasets, as well as the potential inclusion of alternative or ensemble modelling approaches to improve reliability — e.g. by integrating complementary "transformer" models such as BERT or RoBERTa (potentially fine-tuned to be security specific) along with GPT-3.5-Turbo (or a more up-to-date variant), and even incorporating rule-based methods, to average or vote on predictions. This strategy could mitigate the impact of model variability, thus resulting in more robust and consistent detection outcomes, ultimately improving the system's generalisability to broader contexts."

F. Ethical Considerations

- **Public Licensing:** For this research, only publicly licensed code and textual data are analyzed.
- **Responsible Disclosure:** In the event actual high-severity vulnerabilities emerge, repository owners are notified following standard security disclosure practices.
- **Data Anonymization:** Any personal or sensitive information present in Git artifacts is removed or blurred in the final dataset to respect privacy.

IV. Results and Analysis

A. Overview of Experimental Setup

*1) **Objectives**:* The primary aim of this research is to develop and evaluate an NLP-based tool for detecting Self-Admitted Security Debt (SASD) from repository artifacts and mapping the detected instances to the Common Weaknesses Enumeration (CWEs) framework.

Specifically, the experiments address two primary research questions (RQs):

- **RQ1:** How can NLP be applied to automatically detect SASD, and;
- **RQ2:** How can NLP classification accurately map detected SASD instances to CWEs.

*2) **Expected Outcomes**:*

- A **significant detection performance improvement** by the NLP-based approach relative to the baseline keyword-based method.
- **Robust quantitative evidence** confirming that the NLP approach not only flags a higher number of SASD instances, but also achieves a mapping accuracy of at least 75% when categorizing them.

*3) **Experimental Conditions**:*

- **Environment:**
  Experiments were conducted using a Python-based experimental pipeline, executed on a machine with the following specifications:
  - **Processor:** Ryzen 5 5600x processor (3.7GHz, 6 cores, 12 threads)
  - **Memory:** 16GB (3200MHz)
  - **Operating System:** Windows 10
  - **Software:** Python's ThreadPoolExecutor for parallel API calls to efficiently process large datasets.
- **Dataset:**
  The dataset is derived from the Maldonado SATD dataset, a manually labelled dataset of SATD from open-source projects by Everton da Silva Maldonado. This dataset was converted into a code file containing comment blocks with embedded metadata (e.g. "# Project" and "# Classification"). The dataset originally contains approximately 62,000 comment blocks. For experimental feasibility, a stratified sampling method (grouping by classification) was applied to obtain a representative and more manageable subset.
- **Key Parameters**
  - **Baseline Detection:** Uses an expanded list of security related keywords (e.g. "security", "vulnerability", "ssl", "tls", etc).
  - **NLP Detection:** Each comment is processed via an API call to the deployed GPT-3.5-Turbo endpoint.
  - **CWE Mapping:** For comments detected as SASD, the pipeline extracts CWE mapping details from the API response.
  - **Concurrency:** 10-20 threads are used to process the sampled data in parallel.

B. Experimental Results

*1) **SASD Detection Performance**:*

- **Baseline Performance**
  The baseline method achieves a nominal 100% precision; however, since recall is extremely low ($\sim 4-5\%$) this is misleading. Consequently, the baseline's F1-Score hovers around 9%, indicating that it misses the vast majority of SASD instances, although it rarely flags false-positives.
- **NLP Performance**
  In contrast, the NLP-based approach demonstrates near-perfect recall (95-100% across runs) at the expense of

reduced precision ($\sim 43-48\%$). However this trade-off results in a significant improvement in F1-Score ($\sim 60-64\%$), confirming that the NLP approach is far more effective in detecting SASD-related comments.

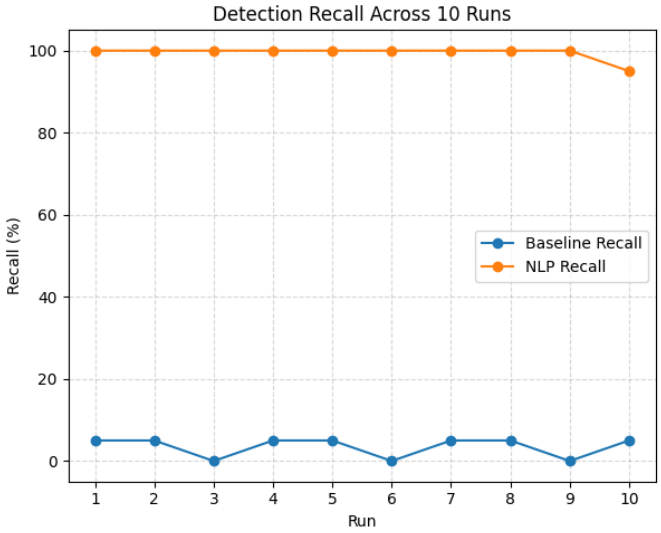

Fig. 1. Baseline and NLP Recall across 10 runs

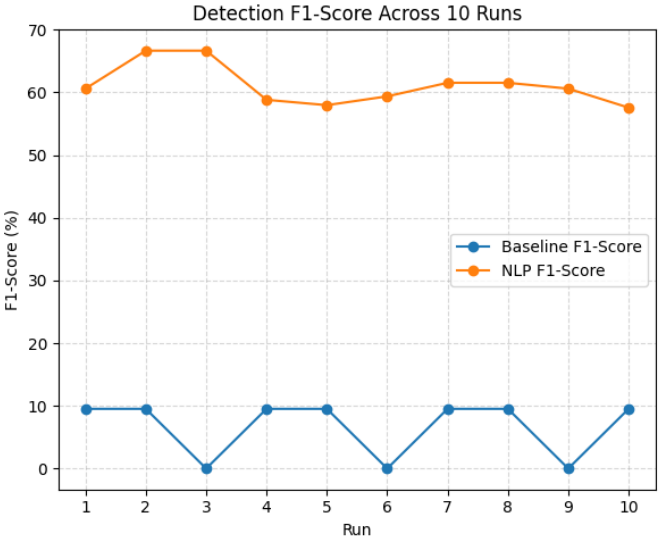

Fig. 2. Baseline and NLP F1-Score across 10 runs

- **Dataset Imbalance Considerations**
  Despite both methods showing a similar overall accuracy ($\sim 10-11\%$), this metric is rendered non-discriminative due to imbalanced class distribution. The true strength of the NLP approach is shown through its ability to detect almost all true-positives, a critical requirement in security-critical contexts.

*2) **CWE Mapping Outcomes**:* CWE mapping outcomes reveal a stark contrast and clear advantage of the NLP-based approach over the baseline, which has an average mapping accuracy of 0%. Across 10 runs, the NLP-based approach reached an average accuracy of 35.4%, with observed values

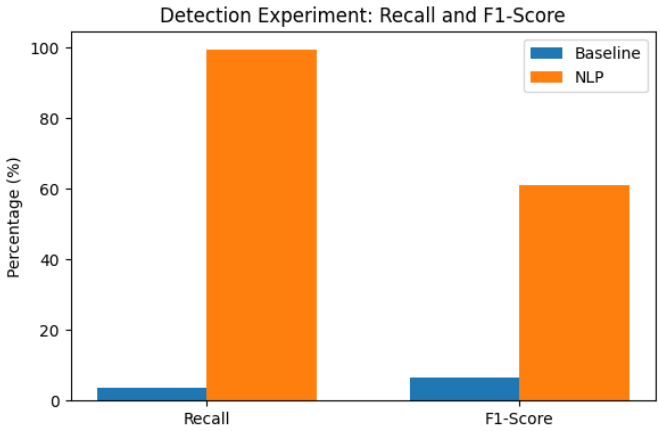

Fig. 3. Baseline and NLP Recall and F1-Score across 10 runs

ranging from 25% to 50%. These trends highlight the substantial discrepancy between the two approaches, but also the consistency of the NLP method despite some variation. This mapping performance is important for effective vulnerability management, as accurate CWE categorisation allows developers to prioritise remediation efforts based on standardised security classifications, thus improving practical management of security debt.

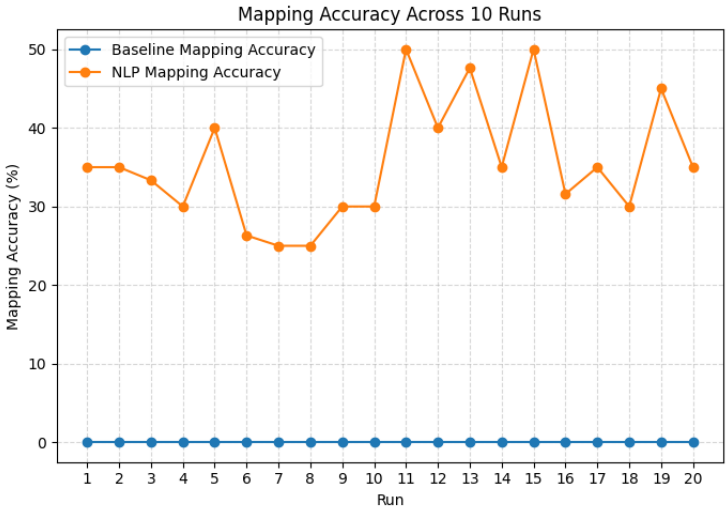

Fig. 4. CWE Mapping Accuracy across 10 runs

*3) **Comparative Evaluation with Existing Studies**:* In their study, Li et al. [11] utilise issue data from several open-source projects using the Jira issue tracking system to train and evaluate their machine learning models for detecting self-admitted technical debt. Upon comparing our findings with those report by Li et al., we observe a consistent trend where ML-based approaches significantly outperform static keyword-based methods for detecting self-admitted debt. Li et al. demonstrated that their approach achieved an F1-score of 0.686 compared to a mere 0.044 for keyword-based detection, a gap that mirrors our own findings where the baseline method, despite its perfect precision, suffers from extremely low recall ($\sim 4-5\%$) and an F1-score of around 9% [11].

In contrast, our NLP-based detection approach attains an F1-score of approximately 60%, illustrating its superior capability in capturing nuanced, contextual indicators of security debt. Furthermore, Li et al. emphasized that even a small training dataset can yield strong performance, which supports our strategy of using synthetic data augmentation to mitigate class imbalance. While Li et al.'s work focused on SATD detection in issue tracking systems, our research focuses on targeting Self-Admitted Security Debt in GitHub repository artifacts, therefore reinforcing the scalability and adaptability of advanced NLP techniques over traditional, rule-based tools like Jira or SonarQube.

C. Analysis and Discussion

1) ***Significance of Findings****:* The practical significance of our findings is multi-faceted, having an impact on both identification and management of security debt in software projects. Our results demonstrate that the NLP-based detection approach not only dramatically improves recall and F1-score over the baseline keyword-based approach, but also performs consistently across multiple experimental runs. This means that in real-world software projects, where the cost of undetected security vulnerabilities can be expensive, an NLP-driven solution has the potential to substantially reduce risks by capturing virtually all instances of self-admitted security debt (SASD). Moreover, although the CWE mapping accuracy of our tool's NLP classifier averages 35.4% — which may at first seem low — it represents a significant and actionable improvement over the baseline approach which achieved 0%. This performance supports automated categorisation of vulnerabilities into standardised CWE frameworks, which further supports developers in prioritising remediation efforts. Ultimately, the integration of advanced NLP techniques into existing development workflows can transform manual, error-prone processes into automated, scalable solutions that improve software resilience.

2) ***Data Interpretation****:* The experimental data reveals trends and trade-offs that are essential for understanding the strengths and limitations of our approaches. The baseline approach, while achieving perfect precision, falls drastically short on recall ($\sim 4-5\%$), thus highlighting a fundamental flaw in relying on static keyword-based detection approaches for security-critical applications. This disparity is especially evident when considering the low F1-score for the baseline method, highlighting that high precision alone is insufficient when the majority of true security debt instances are overlooked. In contrast, the NLP-based detection exhibits near-perfect recall across runs ($\sim 95-100\%$) and a significantly improved F1-score ($\sim 60-64\%$), clearly indicative that this method successfully captures subtle, context-dependent indicators of SASD that simple rule-based methods miss. Furthermore, the mapping component of our study shows that while the average NLP mapping accuracy ranges between $25\% - 50\%$, this consistent improvement over the baseline method offers a solid starting point for automated security vulnerability classification. The variability observed in the mapping accuracy can be attributed to inherent dataset imbalances, NLP model response variability, and the complexity involved in accurately translating nuanced security debt instances into specific CWE categories. These insights reveal that while traditional tools may suffice for detecting obvious issues, advanced NLP methods offer a superior framework for a proactive, scalable, and comprehensive approach to manage Self-Admitted Security Debt in software systems.

3) ***Implications for Practice****:* The practical implications of these findings extend into the daily software development and security management practices. By integrating the developed, NLP-based tool into existing development workflows, development teams can automatically flag and prioritise SASD, thus addressing security vulnerabilities earlier in the development lifecycle. This context-aware approach reduces reliance on manual analysis and static tools such as SonarQube and Jira — which, as previously noted, fail to capture the subtle and contextual nature of self-admitted security debt. Our results suggest that adopting NLP techniques improves detection metrics and offers a scalable framework that could help transform vulnerability management, thus leading to a more proactive security practices and improved software resilience.

D. Summary of Key Findings

In summary, our experiments show that the NLP-based approach offers substantial improvements over traditional keyword-based approaches to detect and categorize SASD. The NLP method achieves near-perfect recall and significantly higher F1-scores, ensuring comprehensive detection of SASD instances, while also yielding robust, albeit modest, improvements in CWE mapping accuracy compared to the baseline. These findings highlight the role of NLP techniques in enabling proactive security management and the practical value of integrating automated detection tools into modern software development workflows.

V. Conclusion

This research has successfully demonstrated the feasibility of using Natural Language Processing (NLP) to detect Self-Admitted Security Debt (SASD) in software projects. The developed NLP-based detection solution significantly outperformed the traditional keyword-based baseline approach, achieving almost perfect recall and substantially improved F1-scores, thus ensuring comprehensive identification of SASD instances. This addresses the limitations noted by Ferreyra et al. [2], who noted that existing "secret scanning tools should extend their scope with SSATD identification features to assist developers". Furthermore, the developed solution showed promising potential in mapping identified SASD instances to Common Weakness Enumerations (CWEs), achieving a noteworthy mapping accuracy improvement compared to the baseline approach. Mock et al. [9] previously proposed using NLP to detect vulnerabilities and related SATD conceptually, this research moves past prototypes stages to empirical

validation, and addresses their identified gap of lacking a comprehensive empirical evaluation.

Despite a moderate mapping accuracy (averaging around 35%), the findings confirm that automated NLP approaches can provide substantial, actionable benefits for prioritising and addressing vulnerabilities, thus addressing concerns raised by Huopio [1] about the potential liability posed by unaddressed security debt.

There are a few areas of the research that could benefit from further improvement or exploration. Firstly, the dataset used for experiments presented limitations with regards to the proportion and representativeness of security-specific debt entries. Addressing this could involve exploring additional datasets that offer greater diversity in security contexts and contemporary software development practices, an issue also identified by Maldonado et al. [4]. Although, given the largely unexplored nature of self-admitted security debt, addressing this issue could prove difficult. Secondly, while GPT-3.5-Turbo provided strong contextual comprehension, further refinement in prompt engineering could further improve precision and consistency, especially in categorising identified SASD instances into appropriate CWE categories. Additionally, experimentation with alternative NLP models (e.g. fine-tuned versions of BERT or RoBERTa), as suggested by Maldonado et al. [4], may improve detected accuracy and model robustness, mitigating the variability in responses seen with GPT-based approaches.

Future research directions should focus on addressing these limitations by incorporating a wider range of datasets, further refining NLP prompt engineering techniques, and rigorously testing various NLP models or ensemble methods to optimize detection accuracy and reliability. Possible integration with established code analysis tools, such as Jira or SonarQube, could also be investigated to potentially advertise for broader adoption of the developed solution in industry workflows. By expanding the research scope in these directions, future studies could significantly improve the capability of automated SASD detection tools, effectively fulfilling the gaps identified by Rindell et al. [5], who emphasised the frequent overlook of security debt during development and testing phases. These advancements would support developers in proactively and efficiently preventing software security risks.

ACKNOWLEDGEMENT

The authors would like to thank Dr. Desmond Greer for his invaluable support, guidance, and insightful and thought-provoking feedback throughout the duration of this project. Dr. Greer's expertise and vision were essential in overcoming the challenges faced and questions raised during the research and development process. His perspective served as a fresh analytic lens that not only addressed immediate issues but also opened up new lines of inquiry, encouraging deeper exploration of potential solutions.

REFERENCES

[1] S. Huopio, "A Quest for Indicators of Security Debt," *The Cyber Defense Review*, vol. 5, no. 1, pp. 169–184, 2020, international Conference on Cyber Conflict (CyCon U.S.) November 18-20, 2019: Defending Forward (Spring 2020). [Online]. Available: https://www.jstor.org/stable/10.2307/26902669

[2] D. Ferreyra, M. Shahin, M. Zahedi, S. Quadri, and R. Scandariato, "What can self-admitted technical debt tell us about security? a mixed-methods study," vol. 2019, pp. 704–715, Apr. 2024.

[3] C. Seaman, Y. Guo, C. Izurieta, Y. Cai, N. Zazworka, F. Shull, and A. Vetrò, "Using Technical Debt Data in Decision Making: Potential Decision Approaches," in *Proceedings of the IEEE Workshop on Managing Technical Debt (MTD)*. IEEE, 2012, pp. 45–48, accessed: Nov. 13, 2024. [Online]. Available: https://ieeexplore-ieee-org.qub.idm.oclc.org/stamp/stamp.jsp?tp=&arnumber=6225999

[4] S. Maldonado, E. Shihab, and N. Tsantalis, "Using natural language processing to automatically detect self-admitted technical debt," *IEEE Transactions on Software Engineering*, vol. 43, no. 11, pp. 1044–1062, Jan. 2017.

[5] K. Rindell, K. Bernsmed, and M. Jaatun, "Managing security in software or: How i learned to stop worrying and manage the security technical debt," 08 2019.

[6] D. Greer, Y. Yu, A. O'Kane, and B. Nuseibeh, "MoRSe: Modelling and refactoring security debt," 2025, grant proposal for research at Queen's University Belfast.

[7] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proceedings of the International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2014.

[8] T. Fahmawi, A. Nabot, I. Jebreen, and A. Al-Qerem, "Exploring code vulnerabilities through code reviews: An empirical study on openstack nova," *Journal of Statistics Applications Probability An International Journal*, vol. 13, no. 2, pp. 681–689, 2024.

[9] M. Mock, "Utilization of machine learning for the detection of self-admitted vulnerabilities," 2023, doctoral proposal. [Online]. Available: https://arxiv.org/pdf/2309.15619

[10] J. Wei and K. Zou, "Eda: Easy data augmentation techniques for boosting performance on text classification tasks," 2019, available: http://github.com/jasonwei20/eda_nlp.

[11] L. Yikun, M. Soliman, and P. Avgeriou, "Identifying self-admitted technical debt in issue tracking systems using machine learning," *arXiv preprint arXiv:2202.02180*, 2022. [Online]. Available: https://arxiv.org/abs/2202.02180